

— Databases

Hoofdstuk 9: Views, functies en stored procedures

L. Espeel



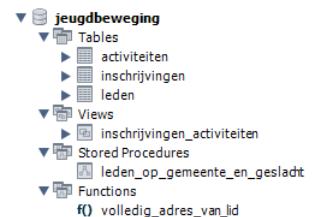
— Queries hergebruiken

Het is mogelijk om veelgebruikte queries, berekeningen of bewerkingen op te slaan in de databank zodat je ze later eenvoudig kunt hergebruiken.

MySQL biedt hiervoor verschillende soorten databaseobjecten, elk met hun eigen kenmerken en toepassingen:

- Views
- Functies
- Stored procedures

Na het aanmaken vind je deze objecten terug onder de respectievelijke categorieën in MySQL Workbench:



— View (1)

Is een opgeslagen SELECT-query die je kunt behandelen als een virtuele tabel. Het bestaat niet echt fysiek, maar maakt het makkelijker om complexe queries herbruikbaar te maken.

Aanmaak:

```
CREATE VIEW inschrijvingen_activiteiten AS
  SELECT i.lidnr, a.omschrijving, a.datumtijdstip
  FROM inschrijvingen i
  JOIN activiteiten a ON i.activiteit_id = a.activiteit_id
  ORDER BY a.datumtijdstip DESC, a.omschrijving;
```

3

— View (2)

Gebruik:

```
SELECT *
FROM inschrijvingen_activiteiten;
```

lidnr	omschrijving	datumtijdstip
1	Kerstfeestje	2015-12-25 15:00:00
7	Kerstfeestje	2015-12-25 15:00:00
11	Kerstfeestje	2015-12-25 15:00:00
13	Kerstfeestje	2015-12-25 15:00:00
16	Kerstfeestje	2015-12-25 15:00:00

```
SELECT l.naam, ia.omschrijving, ia.datumtijdstip
FROM inschrijvingen_activiteiten ia
JOIN leden l ON ia.lidnr = l.lidnr;
```

naam	omschrijving	datumtijdstip
Maxime Debeertst	Kerstfeestje	2015-12-25 15:00:00
Robbe Desnoeck	Kerstfeestje	2015-12-25 15:00:00
Seppe Dewaele	Kerstfeestje	2015-12-25 15:00:00
Jens Cottignie	Kerstfeestje	2015-12-25 15:00:00
Glenn Bruyneel	Kerstfeestje	2015-12-25 15:00:00

4

— Functie (1)

- Wordt gebruikt om berekeningen of bewerkingen uit te voeren die één enkele waarde opleveren.
- Gedraagt zich vergelijkbaar met ingebouwde functies zoals *CONCAT()* of *LENGTH()*.
 - Kan gebruikt worden in een *SELECT*-regel, in een *WHERE*-conditie of een *ORDER BY*.
- Kan nul of meerdere parameters bevatten.
- Moet altijd precies één waarde retourneren.
- Mag de databank niet wijzigen: *INSERT*, *UPDATE*, *DELETE* of *DDL*-queries zijn niet toegestaan.
- Het kan complexere logica bevatten zoals *IF ... THEN ... ELSE ... END IF* en lussen (*WHILE*, *REPEAT* en *LOOP*).
 - (Meer info op: <https://dev.mysql.com/doc/refman/8.0/en/flow-control-statements.html>)

5

— Functie (2)

Aanmaak:

```
DELIMITER $$  
  
CREATE FUNCTION volledig_adres_van_lid(lid_id INT)  
RETURNS VARCHAR(100) DETERMINISTIC  
  
BEGIN  
    DECLARE adresgegevens VARCHAR(100);  
  
    SELECT CONCAT(adres, ", ", postnummer, " ", gemeente)  
    INTO adresgegevens  
    FROM leden  
    WHERE lidnr = lid_id;  
  
    RETURN adresgegevens;  
END$$  
  
DELIMITER ;
```

Binnen een functie (of procedure) bevatten meerdere statements puntkomma's. MySQL zou hierdoor de definitie te vroeg beëindigen.

Door tijdelijk de statement delimiter te wijzigen (bv. naar \$\$) wordt alles tot \$\$ als één statement behandeld, ongeacht de puntkomma's binnen de functie.

Zorg dat het datatype van de parameter overeenstemt met het datatype van argument bij het aanroepen!

DETERMINISTIC: de functie geeft altijd exact hetzelfde resultaat voor dezelfde invoerparameters.

NOT DETERMINISTIC: de functie kan verschillende resultaten teruggeven voor dezelfde invoer, bv. bij gebruik van *NOW()* of *RAND()*.

6

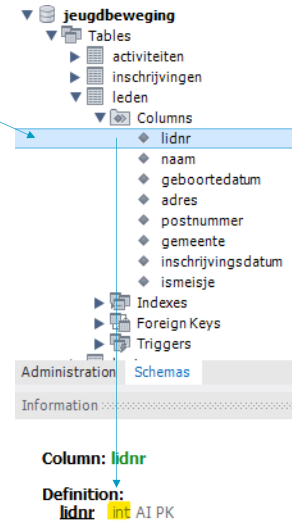
— Functie (3)

Gebruik:

```
SELECT naam, volledig_adres_van_lid(lidnr)
FROM leden;
```

naam	volledig_adres_van_lid(lidnr)
Maxime Debeertst	Dorpstraat 2, 8790 Waregem
Marie Demuytere	Markt 17, 8790 Waregem
Ona Derycke	Vijfseweg 45, 8790 Waregem
Iris Desmet	Stationsstraat 77a, 8790 Waregem
Arthur De Smeytere	Stormestraat 12, 8790 Waregem
Daphne Dierckx	Kerkhofweg 22, 8790 Waregem

Tip: het datatype van een veld kan je op deze manier snel bekijken in MySQL Workbench.



7

— Stored procedure (1)

- Wordt gebruikt om meerdere SQL-instructies uit te voeren.
- Wordt aangeroepen via **CALL**.
- Kan nul of meerdere parameters bevatten.
- Kan nul of meerdere waarden of result sets teruggeven.
- Mag de databank wijzigen: **INSERT**, **UPDATE**, **DELETE** of **DDL**-queries.
 - Afhankelijk van de situatie is het zelfs soms gebruikelijk om in een stored procedure een **TCL-transactie (Transaction Control Language)** te voorzien met daarin meerdere schrijfqueries. Lukken alle schrijfqueries, dan wordt een **commit** uitgevoerd; faalt er echter één schrijfquery, dan wordt een **rollback** uitgevoerd en worden alle eerdere wijzigingen aan de databank ongedaan gemaakt. (Meer info op: <https://dev.mysql.com/doc/refman/8.0/en/commit.html>)
- Kan complexere logica bevatten zoals **IF ... THEN ... ELSE ... END IF**, lussen (**WHILE**, **REPEAT**, **LOOP**) en lokale variabelen.
- Kan worden aangeroepen vanuit applicaties of andere SQL-instructies en kunnen ook records teruggeven die direct in een script gebruikt kunnen worden.

8

— Stored procedure (2)

Aanmaak:

DELIMITER \$\$

```
CREATE PROCEDURE leden_op_gemeente_en_geslacht(gemeente1 VARCHAR(30), ismeisje1 TINYINT)
BEGIN
    SELECT *
    FROM leden
    WHERE gemeente = gemeente1 AND ismeisje = ismeisje1;
END$$

DELIMITER ;
```

9

— Stored procedure (3)

Gebruik:

```
CALL leden_op_gemeente_en_geslacht("Waregem", 1);
```

lidnr	naam	geboortedatum	adres	postrummer	gemeente	inschrijvingsdatum	ismeisje
3	Marie Demuytere	1998-11-02	Markt 17	8790	Waregem	2015-12-17 17:46:27	1
4	Ona Derycke	1998-10-03	Vijvseweg 45	8790	Waregem	2015-12-17 17:47:28	1
5	Iris Desmet	1998-09-04	Stationsstraat 77a	8790	Waregem	2015-12-17 17:48:19	1
9	Clémence Desmet	1998-06-16	Processiestraat 69	8790	Waregem	2015-12-17 17:52:12	1

10

— **Stored procedure** ⁽⁴⁾

Een nuttige case om zelf eens uit te proberen ...

Maak een stored procedure die een activiteit uit de database verwijdert op basis van een opgegeven activiteit in de tabellen *activiteiten* en *inschrijvingen*.

- Er zullen hierin twee DELETE-queries nodig zijn, zie hoofdstuk 8.

```
CALL verwijder_activiteit("Halloweentocht");
```